

**Projecto de Arquitectura de Computadores Ano
lectivo 2025/2026**

Departamento de Ensino e Investigação de Ciências da Computação Faculdade
de Ciências Naturais da Universidade Agostinho Neto

Pac-Man Simplificado

Grupo Nº 12

Integrantes do Grupo

Hélvio Santana(Não faz parte do classroom)

Github: helviocelso234@outlook.com

Contacto: 937326780

Goldwyn Narciso

Github: goldwynnarciso43@gmail.com

Contacto: 932731390

Hamilton Zua

Github: hamiltonmanuel019@gmail.com

Contacto: 942719458

André Gunza(Não faz parte do classroom)

Github: andreluambagunza@gmail.com

Contacto: 922414879

Endereço do repositório: <https://github.com/helviocelso456/PAC-MAN-AC>

Docente: João José da Costa

Sumário

Introdução	3
Concepção e Implementação	4
Estrutura Geral	4
Mapa de endereçamento escolhido	5
Comunicação entre processos	5
Variáveis de Estado	5
Interrupções.....	6
Rotinas	6
Conclusões	7
Código assembly	8

Introdução

Este relatório descreve o desenvolvimento de uma versão simplificada do jogo *Pac-Man* em linguagem Assembly para o processador PEPE-16, realizado no âmbito da disciplina de Arquitetura de Computadores. O projeto foi implementado no simulador desenvolvido em Java pelo docente João José da Costa, que fornece um ambiente de execução com suporte a I/O mapeado em memória, display gráfico e teclado.

Contexto do trabalho: O trabalho foi desenvolvido para a plataforma PEPE-16, um processador didático de 16 bits. O simulador em Java permite a execução de programas Assembly PEPE-16, com acesso a periféricos como teclado matricial e display de pixels.

Objectivos:

1. Implementar um jogo Pac-Man funcional com movimentação do jogador e fantasmas.
2. Utilizar o teclado matricial para controlar a direção do Pac-Man.
3. Implementar um sistema de colisões com fantasmas e objetos colecionáveis.
4. Gerir vidas, pontuação e condições de vitória/derrota.
5. Desenhar o estado do jogo no display de pixels, incluindo sprites e bordas coloridas.

Restrições e Observações: O display do PEPE-16 utiliza uma matriz de pixels com suporte a cores (vermelho e preto/branco). O jogo foi implementado com sprites de 3x3 pixels para representar o Pac-Man, fantasmas, objetos e a caixa central. A inteligência dos fantasmas é baseada em perseguição ao Pac-Man, com movimentos lentos e atrasos controlados por software.

Na secção 2 descreve-se a conceção e implementação do sistema, incluindo a estrutura geral, mapa de endereçamento, comunicação entre processos, variáveis de estado, interrupções e rotinas principais. Na secção 3 apresentam-se as conclusões, e na secção 4 o código assembly completo.

Mapa de endereçamento escolhido

- 8000H a 807FH: Memória de vídeo para pixels pretos/brancos (128 bytes, 32x32 bits).
- 9000H a 907FH: Memória de vídeo para pixels vermelhos (camada de cor).
- 0A000H: Porto dos displays hexadecimais (para mostrar o contador de segundos).
- 0C000H: Porto de saída do teclado (para selecionar linha).
- 0E000H: Porto de entrada do teclado (para ler coluna).
- 4000H: Buffer de tecla (armazena a tecla pressionada).

Comunicação entre processos

Não há múltiplos processos no sentido tradicional, mas o sistema é baseado em um ciclo principal que chama rotinas específicas. A comunicação entre o teclado e o jogo é feita através do buffer 4000H, que é atualizado pela rotina pTeclado e lido pelo ciclo principal.

Variáveis de Estado

As principais variáveis de estado definidas são:

- linha_pac, coluna_pac: Coordenadas do Pac-Man.
- fantasma_linha, fantasma_coluna: Coordenadas do primeiro fantasma.
- fantasma2_linha, fantasma2_coluna: Coordenadas do segundo fantasma (ativado após 1 segundo).
- vidas: Número de vidas restantes (inicializado a 3).
- pontuacao: Pontuação atual (não implementada na versão final, mas definida).
- objetos_coletados: Contador de objetos coletados (0-4).
- objeto_0 a objeto_3: Estados dos objetos nos cantos (0 = não coletado, 1 = coletado).
- game_active: Flag que indica se o jogo está ativo (1) ou terminou (0).
- contador_decimal: Contador de segundos (0-99) mostrado nos displays hexadecimais.

Interrupções

O sistema não utiliza interrupções de hardware. O teclado é lido através de *polling* na rotina pTeclado, que varre as linhas e colunas do teclado matricial e armazena a tecla pressionada no buffer 4000H. A atualização do display é feita diretamente pela escrita nos endereços de memória de vídeo.

Rotinas

main_loop: Ciclo principal que chama sequencialmente:

- pTeclado: Lê o teclado e armazena a tecla no buffer.
- Processa a tecla (movimento do Pac-Man).
- mover_fantasma_lento: Move o primeiro fantasma (a cada 2 ciclos).
- mover_fantasma2_lento: Move o segundo fantasma (a cada 3 ciclos, se ativo).
- verificar_colisao e verificar_colisao2: Verificam colisões com os fantasmas.
- verificar_colisao_objetos: Verifica colisões com os objetos dos cantos.
- verificar_vitoria_global: Verifica se todos objetos foram coletados.
- atualizar_contador_segundos: Atualiza o contador de segundos nos displays.
- ativar_segundo_fantasma: Ativa o segundo fantasma após 1 segundo.
- delay: Introduz um atraso para controlar a velocidade do jogo.

Rotinas de desenho:

- pixel_xy: Ativa um pixel na camada preta/branca.
- desenhar_pixel_vermelho: Ativa um pixel na camada vermelha.
- desenhar_sprite_3x3: Desenha um sprite de 3x3 pixels.
- apagar_sprite_3x3: Apaga um sprite de 3x3 pixels.

Rotinas de jogo:

- mover_pac_cima, mover_pac_baixo, mover_pac_esquerda, mover_pac_direita: Movem o Pac-Man.
- mover_fantasma e mover_fantasma2: Movem os fantasmas em direção ao Pac-Man (perseguição).
- perder_vida: Decrementa vidas e reposiciona o Pac-Man após colisão.
- vitoria e mostrar_game_over: Mostram ecrãs de fim de jogo.

Conclusões

Objectivos iniciais: O projeto visava a criação de um jogo Pac-Man jogável em Assembly PEPE-16, com todos os elementos básicos: labirinto (bordas vermelhas), Pac-Man controlável, dois fantasmas perseguidores, objetos colecionáveis e sistema de vidas.

Concretizado: Foi possível implementar com sucesso a maioria das funcionalidades. O Pac-Man move-se com fluidez através do teclado matricial, os fantasmas perseguem o jogador de forma eficaz, as colisões são detectadas corretamente, os objetos dos cantos são coletados, e o contador de segundos é atualizado nos displays. A ativação do segundo fantasma após 1 segundo funciona conforme planeado.

Críticas e melhorias: A pontuação não foi implementada na versão final, apesar de ter uma variável definida. A velocidade do jogo é controlada por um delay fixo, o que pode tornar o jogo muito lento ou rápido dependendo do desempenho do simulador. Seria interessante implementar níveis de dificuldade e mais fantasmas.

Funcionamento geral: A solução apresentada é estável e cumpre os requisitos principais. As opções de projeto, como o uso de sprites 3x3, a camada vermelha para bordas e a perseguição dos fantasmas, mostraram-se adequadas e eficazes.

Sugestões de melhoramento:

1. Implementar a pontuação e mostrá-la nos displays ou no display de pixels.
2. Adicionar mais tipos de objetos (power-ups) que afetem o comportamento dos fantasmas.
3. Implementar sons através do simulador (se suportado).
4. Criar um menu inicial e vários níveis com labirintos diferentes.

Código assembly

O código assembly completo está disponível no ficheiro pacman.asm. Abaixo apresenta-se um resumo da estrutura principal:

```
; =====
; ARQUITECTURA DE COMPUTADORES
; PAC-MAN SIMPLIFICADO PARA O PEPE-16
; GRUPO - 12
; AUTORES: HÉLVIO SANTANA, GOLDWYN NARCISO, HAMILTON ZUA, ANDRÉ GUNZA
; DOCENTE: JOÃO JOSÉ DA COSTA
; =====

; Constantes de endereçamento
BUFFER      EQU 4000H
PIN          EQU 0E000H
POUT         EQU 0C000H
pixelsMatriz EQU 8000H
pixelsVermelho EQU 9000H
displays     EQU 0A000H

; Variáveis de estado
PLACE 3200H
linha_pac:      WORD 15
coluna_pac:     WORD 7
fantasma_linha: WORD 14
fantasma_coluna: WORD 14
; ... (demais variáveis)

; Sprites 3x3
PLACE 3500H
sprite_pacman:  ; Formato "C"
sprite_fantasma: ; Formato "X"
sprite_canto:   ; Formato "+"
sprite_caixa:   ; Caixa central

; Programa principal
PLACE 0
inicio:
    MOV SP, stackBase
    CALL Carregamento

main_loop:
    CALL pTeclado
    ; Processamento do jogo
    ; ... (rotinas de movimento, colisão, etc.)
    JMP main_loop

; Rotinas de desenho
pixel_xy:
    ; Ativa pixel na camada preto/branco
    RET

desenhar_pixel_vermelho:
    ; Ativa pixel na camada vermelha
    RET

; Rotinas de jogo
```

```

mover_pac_cima:
    ; Movimentação do Pac-Man
    RET

mover_fantasma:
    ; Perseguição inteligente ao Pac-Man
    RET

; Rotina de teclado (polling)
pTeclado:
    ; Varredura do teclado matricial 4x4
    RET

; ... (demais rotinas)

```

Nota: O código completo contém aproximadamente mais de 2400 linhas de Assembly para o PEPE-16, incluindo todas as rotinas de desenho, movimentação, colisão, e controlo do jogo. O ficheiro completo está disponível para consulta.